

RFC-0002 — Component Language Philosophy

Status: Draft

Category: Language Design

Version: 0.1

1. Purpose

This document defines the philosophy behind the Component Language.

It intentionally does not define syntax.

Syntax belongs to future specifications.

This RFC defines how the language should think.

2. Why another language?

Electronic design already has:

- Verilog
- SystemVerilog
- VHDL

These languages are excellent for hardware implementation.

However, they are not designed for:

- beginners
- visual construction
- AI interaction
- voice interaction
- educational workflows
- physical DIP components

Component Language exists for those purposes.

3. Design Goal

The language should describe

what the circuit is

rather than

how to execute the circuit

The language is declarative.

4. Component is Structural

A Component describes structure.

It does not describe execution order.

Instead it describes:

- objects
 - relationships
 - hierarchy
 - connectivity
-

5. Component is Functional

The language describes functions.

Examples:

Counter

Register

Memory

Clock

Bus

It does not describe procedural execution.

A Register knows internally that it captures data on a clock edge.

The language should not require users to write procedural hardware descriptions.

6. Component is Physical

Unlike traditional HDLs,

Component understands real integrated circuits.

Examples:

74HC161

74HC245

AT28C256

62256

Pins are physical objects.

Packages are physical objects.

DIP pin numbers remain important.

7. Component is Human Language

The language should read like engineering English.

Example concepts:

connect

chip

input

output

group

bus

probe

watch

Students should understand a design before learning HDL syntax.

8. Component is Voice Friendly

Every construct should be expressible naturally.

Example:

"Connect Clock to Counter CLK."

rather than

"Instantiate module..."

Voice interaction is a first-class design goal.

9. Component is AI Friendly

The language should be deterministic.

Every statement should have exactly one meaning.

Avoid ambiguous shorthand.

Avoid hidden behavior.

Every statement should be easily transformed into structured operations.

10. Component is Visual

Every language construct should have a visual representation.

Likewise,

every visual manipulation should be representable in the language.

Visual and text are equal citizens.

Neither is primary.

11. Component is Package Friendly

Everything needed to describe circuit meaning belongs to Component.

Everything needed to describe presentation belongs to Board.

Everything needed to describe dependencies belongs to Resources.

12. Component is Git Friendly

Small changes should produce small textual differences.

Example:

Changing

LED1

to

LED2

should produce a one-line diff.

The language should avoid unnecessary reformatting.

13. Component is Stable

Language meaning should remain stable across future versions.

New capabilities should extend rather than redefine the language.

Backward compatibility is preferred.

14. Component borrows from SystemVerilog

Component intentionally adopts ideas from SystemVerilog.

Examples include:

Modules

Named ports

Typed ports

Vectors

Interfaces

Hierarchy

The language intentionally avoids unnecessary complexity.

Examples not intended for the initial language include:

Procedural blocks

Always blocks

Fork/Join

Classes

Randomization

Assertions

Coverage

Verification constructs

15. Component is Educational

The language should encourage understanding.

A beginner should be able to read a project.

Even if they cannot write one.

16. Component is Explainable

Every statement should answer

"What does this mean?"

rather than

"What does this execute?"

This enables AI explanation.

17. Component is Package Independent

The language does not know:

Flutter

Qt

React

Desktop

Tablet

Browser

VR

The language belongs to the Platform.

18. Component is Future Proof

The language should remain useful even if future Boards become

3D

AR

VR

Robotics

Printed documentation

Voice-only

AI-generated

The language remains unchanged.

19. Component is Canonical

Component is the canonical description of circuit meaning.

Visual Boards

Generated Verilog

Generated SystemVerilog

Simulation

Documentation

All derive from Component.

20. Design Rules

The language should be:

Simple

Readable

Minimal

Explicit

Deterministic

Composable

Hierarchical

Extensible

AI Friendly

Voice Friendly

Student Friendly

Physical

Visual

Portable

21. Non-Goals

Component is not intended to replace:

SystemVerilog

Verilog

VHDL

PCB CAD

SPICE

Instead,

Component complements them.

22. Principle

Component describes meaning.

Everything else is a projection.